

Mellow Language Cheat Sheet

Canonical builtins, keywords, libraries, and runtime rules

Mellow Language project

1. Quick syntax

Function calls use angle brackets:

```
print<"Hello, Mellow!">
add<2, 3>
if greater_than<score, 10> [
    print<"high score">
]
```

Blocks use square brackets. Comments use `//` and `/* ... */`.

2. Keywords and control flow `true` / `false` Boolean literals. `null` Null value. `let` / `const` Declare variables. The form `let` creates a variable that starts as `null` unless initialized. `if` / `else` Conditional blocks. `while` / `loop` / `break` / `continue` Repeated execution and control. `for` Iterate over arrays or lists with `for x in collection [ ... ]`. `return` Return a value from a function or method. `func` / `fn` Define a named function. `class` Declare a class with fields and methods. `public` / `private` / `static` Visibility and class-level data. `import` Load declarations from another Mellow file. `try` / `catch` Handle runtime errors. `catch<>` catches all; `catch<ValueErr>` catches a specific type. `raise` Raise a runtime error with a message. `this` / `super` Class instance and parent access. `and` / `or` / `not` Boolean operators as named builtins.

3. Builtins: arithmetic `add<a, b>` Adds numbers; concatenates strings or same-kind collections. `subtract<a, b>` Numeric subtraction. `multiply<a, b>` Numeric multiplication. `divide<a, b>` Numeric division. Zero divisor raises `DivisionErr`. `modulo<a, b>` Floating-point remainder. `power<a, b>` Raise `a` to the power `b`. `sqrt<a>` Square root. `inc<a>` Return `a + 1`. `dec<a>` Return `a - 1`.

Deprecated aliases such as `sub`, `mul`, `div`, `mod`, `pow`, `eq`, `lt`, `gt`, `lte`, `gte`, `less`, `greater`, `sum`, `minus`, `times`, `quotient`, and `remainder` are rejected with a `ValueErr` that points to the canonical replacement.

4. Builtins: comparisons and logic `equal<a, b>` True when values are equal. `not_equal<a, b>` True when values differ. `less_than<a, b>` Numeric less-than test. `greater_than<a, b>` Numeric greater-than test. `less_equal<a, b>` Numeric less-than-or-equal test. `greater_equal<a, b>` Numeric greater-than-or-equal test. `and<a, b>` Boolean conjunction. `or<a, b>` Boolean disjunction. `not<a>` Boolean negation. `inside<value, container>` Checks for string membership, array/list membership, or dictionary key presence. `contains` is an alias.

5. Builtins: strings and text `trim<string>` Remove surrounding whitespace. `upper<string>` Uppercase conversion. `lower<string>` Lowercase conversion. `replace<text, old, new>` Replace all occurrences of a substring. `starts_with<text, prefix>` Check a prefix. `ends_with<text, suffix>` Check a suffix. `split<text, delimiter>` Split a string into a list. `join<list, delimiter>` Join a list of strings with a delimiter. `input<>` Read one line from stdin. `input<"Prompt: ">` Print a prompt, then read input. `to_number<string|number>` Convert text to a number. `to_float<string|number>` Convert to a floating-point number. `to_int<string|number>` Convert and truncate to an integer-valued number. `to_bool<value>` Convert true/false, 1/0, or string booleans.

6. **Builtins: collections and values** **list**<item, item, ...> Create a list. **dict**<> Create an empty dictionary. **get**<container, key> Read a dictionary entry or collection index. **has**<container, key> Check whether a dictionary key or collection index exists. **put**<dictionary, key, value> Add or replace a dictionary entry. **len**<value> String length or collection size. **type**<value> Return the runtime type name. **to_string**<value> Convert a value to text. **range**<start, end[, step]> Create a numeric list ending before end. **set**<name, value> Assign a mutable variable. **print**<value, ...> Print values separated by spaces and finish with a newline. **destroy**<instance> Destroy an instance and call its **free**<> method if present.

7. **Builtins: numeric helpers and predicates** **abs**<a> Absolute value. **floor**<a> Floor value. **ceil**<a> Ceiling value. **round**<a> Rounded value. **min**<a, b> Smaller numeric value. **max**<a, b> Larger numeric value. **clamp**<value, min, max> Clamp to range. **is_null**<value> Check for null. **is_number**<value> Check for number. **is_string**<value> Check for string. **is_array**<value> Check for array. **is_list**<value> Check for list. **is_dict**<value> Check for dictionary.

8. **Error types** **ValueErr** General value or runtime mismatch error. **RecurErr** Recursion-related error. **DivisionErr** Division-by-zero or invalid division path. **SyntaxErr** Syntax or parser-level issue.

A **catch**<error: **ValueErr**> style handler binds a message and filters by type.

9. Collection methods

These methods work on lists and dictionaries: **list.push**<value> Append one item. **list.pop**<> Remove and return the last value. **list.get**<index> Read an item by index. **list.set**<index, value> Replace an item by index. **list.remove**<index> Remove an item by index. **list.clear**<> Empty the list. **list.has**<value> Check whether a list contains the value. **dict.get**<key> Read a dictionary value by string key. **dict.put**<key, value> Set or replace a dictionary entry. **dict.has**<key> Check whether a string key exists. **dict.remove**<key> Remove and return a key/value pair. **dict.keys**<> Return a list of keys. **dict.values**<> Return a list of values. **dict.clear**<> Empty the dictionary.

10. Import and libraries

Mellow loads other files with **import**:

```
import "math.mll" <random>
import "path/to/file.mll" <hello, Parser>
```

The current runtime supports import-based code loading and builtins such as **random**<> and **rand**<>. The active implementation does not require a separate predeclared module namespace for list or dictionary helpers; those are provided by collection builtins and methods.

11. Examples

```
let name = "Mellow"
let total = add<2, multiply<3, 4>>
print<"Total: ", total>

try [
  divide<5, 0>
] catch<err: DivisionErr> [
  print<"division failed: ", err>
]
```

The canonical naming style is long-form and explicit: **add**, **subtract**, **multiply**, **divide**, **modulo**, **equal**, **greater_than**, **less_than**, **greater_equal**, **less_equal**, and **not_equal**. Created for the Mellow language reference and quick lookup.